

✓ Assignment 7 by Chris

House Price Prediction Using Bolton Housing Data

This dataset provides information about house prices in Bolton. House Price Prediction

In this notebook, you'll follow the basic machine learning process to build a regression model to predict house prices using the "Boston Housing Dataset" from sklearn. The regression model will either be a Decision Tree or Random Forest regressor.

Follow the instructions and complete each TODO to complete the assessment on the essential steps in building and evaluating a regression model.

The following is a description of each column in the dataset:

Dataset Features (Bolton Housing):

- CRIM: Crime rate by town
- ZN: Proportion of residential land zoned for large lots
- INDUS: Proportion of non-retail business acres per town
- CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- NOX: Nitric oxide concentration (parts per 10 million)
- RM: Average number of rooms per dwelling
- AGE: Proportion of owner-occupied units built before 1940
- DIS: Weighted distances to five Boston employment centers
- RAD: Index of accessibility to radial highways
- TAX: Full-value property tax rate per \$10,000
- PTRATIO: Pupil-teacher ratio by town
- B: Proportion of Black population
- LSTAT: Percentage of lower status of the population
- MEDV (Target): Median value of owner-occupied homes in \$1,000s

Dataset is from sklearn Datasets

Link to github

https://github.com/christones/williscollege/blob/main/Lesson_7_assignment.ipynb

◆ Gemini

```
# --- Imports ---  
# TODO: Import all the necessary libraries for data handling, visualization, and mod
```

```
# Example: import pandas as pd
# Add your imports here:
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import tensorflow as tf # Import tensorflow for dataset loading

# --- Data Collection and Loading ---
# TODO: Load the 'Boston Housing' dataset from sklearn and convert it into a pandas
# Hint: Use `load_boston()` from `sklearn.datasets`

# Load dataset and convert to DataFrame:

# Add your code here:
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

df = pd.DataFrame(data, columns=[
    'CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO',
])
df['MEDV'] = target
# Using tf.keras.datasets.boston_housing to load the dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.boston_housing.load_data()

# Combine train and test data to form a single dataset similar to original load_boston
X = np.vstack([x_train, x_test])
y = np.concatenate([y_train, y_test])

# Define feature names as they are not directly provided by keras loader
feature_names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO']

df = pd.DataFrame(X, columns=feature_names)
df['MEDV'] = y

# --- Quick Check of Data ---
# TODO: Display the first few rows of the dataset to understand its structure.
# Hint: Use `.head()` to inspect the first few rows.

# Add your code here:
df.head()

# TODO: Check the features and target variable. Identify which is continuous and categorical.
# Hint: Use `.info()` and `.describe()` to inspect data types and statistical properties.

# Add your code here:
df.info()
df.describe()
```

```
# --- EDA and Data Preprocessing ---
# TODO: Check for missing/null values.
# Hint: Use `.isnull().sum()` to check for null values.

# Add your code here:
df.isnull().sum()

# TODO: Visualize the data. Create scatter plots to see the relationship between ind
# Example: Use `plt.scatter()` to visualize the relationship between features like

# Add your code here:
plt.figure(figsize=(8, 5))
plt.scatter(df['RM'], df['MEDV'], alpha=0.5)
plt.xlabel('Average Number of Rooms (RM)')
plt.ylabel('Median House Value (MEDV)')
plt.title('RM vs MEDV')
plt.show()

# TODO: Create a function to automate scatter plots for all features vs the target v
# Hint: The function should loop over a list of features and plot scatter plots for

# Define your function here:
def plot_features_vs_target(df, features, target):
    for feature in features:
        plt.figure(figsize=(6, 4))
        plt.scatter(df[feature], df[target], alpha=0.5)
        plt.xlabel(feature)
        plt.ylabel(target)
        plt.title(f'{feature} vs {target}')
        plt.show()

# TODO: Use the function to visualize the relationships between multiple features an
# Example: ['RM', 'LSTAT', 'AGE', 'CRIM']
# Target: 'MEDV'

# Add your code here:
features_to_plot = ['RM', 'LSTAT', 'AGE', 'CRIM']
plot_features_vs_target(df, features_to_plot, 'MEDV')

# --- ML Model Training ---
# TODO: Split the dataset into training and testing sets.
# Hint: Use `train_test_split()` from `sklearn.model_selection` with an 80/20 split.

# Define X (features) and y (target) and perform the train-test split:
X = df.drop('MEDV', axis=1)
y = df['MEDV']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# TODO: Choose an appropriate regression model: Decision Tree or Random Forest.
# Hint: Use either `DecisionTreeRegressor` or `RandomForestRegressor` from `sklearn`.
```

```
# Define your regression model here:
model = RandomForestRegressor(random_state=42)

# TODO: Train the model on the training data.
# Hint: Use `.fit()` to train the model.

# Add your code here:
model.fit(X_train, y_train)

# --- Model Evaluation ---
# TODO: Evaluate the performance of the model on the test set using relevant metrics
# Hint: Use `mean_squared_error()` and `r2_score()` from `sklearn.metrics`.

# Perform parameter tuning on the model if needed to improve the performance of your

# Add your parameter tuning code here:
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)

# Use the best model from the grid search
best_model = grid_search.best_estimator_

# Predict on the test set and calculate the evaluation metrics:
y_pred = best_model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.4f}")
print(f"R-squared: {r2:.4f}")

# --- Model Prediction ---
# TODO: Predict house prices from a new set of feature inputs.
# Example new data: Use hypothetical or randomly generated values for the features.
#
# Example new data: CRIM = 0.2, ZN = 12.5, INDUS = 7.07, CHAS = 0, NOX = 0.5, RM = 6
```

```
# Add your prediction code here:
new_data = pd.DataFrame([{
    'CRIM': 0.2,
    'ZN': 12.5,
    'INDUS': 7.07,
    'CHAS': 0,
    'NOX': 0.5,
    'RM': 6.5,
    'AGE': 68,
    'DIS': 4.0,
    'RAD': 2,
    'TAX': 250,
    'PTRATIO': 17,
    'B': 400,
    'LSTAT': 12
}])

# Ensure column order matches the training features
new_data = new_data[X.columns]

prediction = best_model.predict(new_data)
print(f"Predicted House Price (MEDV): {prediction[0]:.4f} (in $1000s)")
```


Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/bos>

57026/57026 ————— **0s** 0us/step

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 506 entries, 0 to 505

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	CRIM	506 non-null	float64
1	ZN	506 non-null	float64
2	INDUS	506 non-null	float64
3	CHAS	506 non-null	float64
4	NOX	506 non-null	float64
5	RM	506 non-null	float64
6	AGE	506 non-null	float64
7	DIS	506 non-null	float64
8	RAD	506 non-null	float64
9	TAX	506 non-null	float64
10	PTRATIO	506 non-null	float64
11	B	506 non-null	float64
12	LSTAT	506 non-null	float64
13	MEDV	506 non-null	float64

dtypes: float64(14)

memory usage: 55.5 KB



